

31.1 Quicksort Flavors vs. MergeSort

Quicksort Flavors: Motivation

Before we dive into the wonderful ideas of how to improve the Quicksort Algorithm, here's a [review](#) on how the Quicksort (or Partition Sort) works.

Recall, our version of Quicksort has the following properties:

- Leftmost item is always the pivot
- Our partitioning algorithm always preserves the relative order of $\leq$ and $\geq$ items.

And remember that even though if we have a pivot that is always “good”, which leads to a best case runtime of $\theta(N \log N)$, it is still *possible* that the rare worst case runtime $\theta(N^2)$ will happen in practice. Consequently, Quicksort will have a worse runtime than [MergeSort](#), losing its title as the “fastest sorting algorithm”.

Therefore, a natural question to ask is: **What can we do to avoid running into the worst case for Quicksort?** So it can beat MergeSort for sure? This will be the main motivation of exploration for this section.

Quicksort Flavors: Philosophies

Here are some general philosophies to follow to brainstorm ways to improve Quicksort

Philosophy 1: Randomness

Some possible reasons that the rare worst case runtime still happen in practice include:

- Bad ordering: Arrays already in sorted order
- Bad elements: Arrays with all duplicates

In both of the examples above, assuming we are using the strategy that the leftmost item is always the pivot, each partition of the elements around the pivot **fails to reduce the size of the resulting subarrays _significantly_**, which leads to the consequence of going through the items many many times.

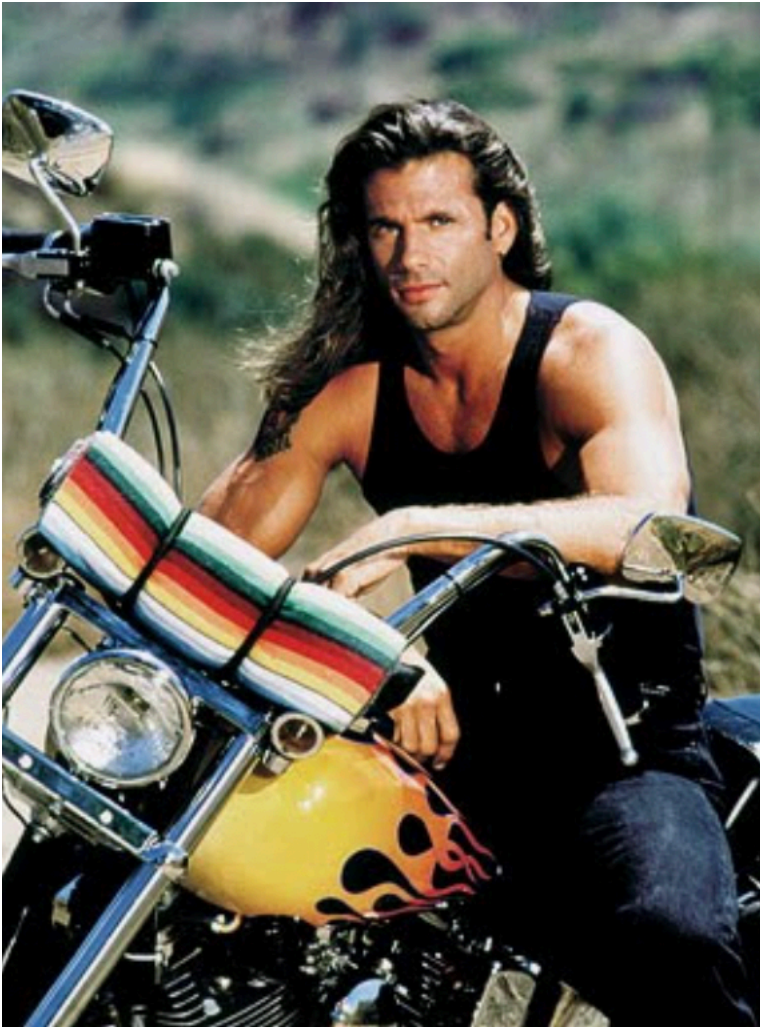
One solution to the bad ordering problem is randomness, which is encoded by these two strategies below:

1. Pick pivots randomly.
2. Shuffle items before sort.

Note: Strategy 2 requires care in partitioning code to avoid $\theta(N^2)$ behavior on arrays of duplicates.

Turns out, this randomized version of Quicksort is the best sort you can get with extremely good performance.

Just like the care-free, motorcycle rider from the 1992 TV Show *Renegade*, this version of the Quicksort algorithm is able to thrive with a devil-may-care attitude, even without the luxurious hair.



Spirit of Randomized Quicksort Algorithm.

Philosophy 2: Smarter Pivot Selection

Another approach to the problem is to choose a “good” pivot effectively, somehow.

2a. Constant time pivot pick

One of such approach is to pick a few items, and then among them, choose the “best” one as the pivot. This type of approach exemplifies a type of pivot selection procedure that is both *deterministic* and *constant* time.

However, a big drawback of such procedure is that the resulting Quicksort has a family of “dangerous inputs”—inputs that will lead to the worst case runtime or break the algorithm—that an adversary could easily generate.

2b. Linear time pivot pick

Since we are partitioning multiple times, always selecting the median as the pivot seems like a good idea, because each partition will allow us to cut the size of the resulting subarrays by half.

Therefore, to improve upon the idea from part a, we can calculate the *median* of a given array, which can be done in linear time, then select that as the pivot.

The “exact median QuickSort” algorithm will not have the technical vulnerabilities as the 2a approach, and has a runtime of $\theta(N \log N)$. However, it is still slower than MergeSort.

Philosophy 3: Introspection

This philosophy simply relies on introspecting the recursive depth of our algorithm, and switch to MergeSort once we hit the depth threshold.

Although this is a reasonable approach, it is not common to use in practice.

Quicksort Flavors vs. MergeSort: Who’s the best?

Candidate One: Quicksort L3S

Quicksort L3S is the Quicksort that is introduced last time, with the following properties:

- Always leftmost pivot selection
- Partitioning
- Shuffle before starting

	Pivot Selection Strategy	Partition Algorithm	V A S
Mergesort	N/A	N/A	N
Quicksort L3S	Leftmost	3-scan	S

Unfortunately, 0 : 1 with Mergesort in the lead.

Candidate Two: Quicksort LTHS

Quicksort LTHS is very similar to Quicksort L3S, except with a different partitioning scheme: [Tony Hoare's In-Place Partitioning Scheme](#).

Tony Hoare's Partitioning

Imagine two pointers, one at each end of the items array, walking towards each other:

- Left pointer loves small items.
- Right pointer loves large items.
- Big idea: Walk towards each other, swapping anything they don't like; stop when the two pointers cross each other.
 - Left pointer hates larger or equal items
 - Right pointer hates smaller or equal items
- New pivot = Item at right pointer.
- End result is that things on left of pivot (originally the leftmost item) are "small" and things on the right of pivot are "large".

TH Partitioning: An Intuitive Analogy

Intuitively, you can think of each pointer as a little "police" for the "less than pivot" territory and "greater than pivot" territory. And think of the pivot as the "border line" between the two territories.

Their job is to find and arrest any "suspects", or items that don't belong to their respective territories, and trade the arrested suspects with the "police" on the other side.

Specifically, the left pointer is the "police" for the "less than pivot" territory, looking for any "suspects" (items greater than or equal to the pivot) that are not supposed to be on this side of the border line. Analogously, the right pointer looks for "suspects" (items less than or equal to the pivot) in the "greater than territory".

When the two "polices" meet, mission is done as they both have sufficiently searched across their representative territories.

All this work for nothing?

Not at all! All this work with a new partitioning scheme did pay off, and using the TH partitioning scheme yields a very fast Quicksort: one that is faster than MergeSort!

It's important to also note that:

- Faster schemes have been found since.
- Overall runtime still depends crucially on pivot selection strategy!

	Pivot Selection Strategy	Partition Algorithm	Worst Case Complexity
Mergesort	N/A	N/A	N/A
Quicksort L3S	Leftmost	3-scan	Shuffle
Quicksort LTHS	Leftmost	Tony Hoare	Shuffle

1 : 1 with a win for QuickSort Flavors!

Candidate Three: QuickSort PickTH

Median Identification: Linear Time

Recall from philosophy 2b that the idea of identifying the median and use it as the pivot is inefficient because finding the median itself is a costly process. (Runtime: $\theta(N \log N)$)

Turns out, it is possible to find the median in $\theta(N)$ time instead, with an algorithm called "[PICK](#)".

Will this improved version of Exact Median Quicksort perform better than Mergesort?

	Pivot Selection Strategy	Partition Algorithm	Worst Case Complexity
Mergesort	N/A	N/A	N/A
Quicksort L3S	Leftmost	3-scan	Shuffle
Quicksort LTHS	Leftmost	Tony Hoare	Shuffle
Quicksort PickTH	Exact Median	Tony Hoare	Exact

Sadly, no. And it was terrible! Likely for the following reasons:

- Cost to compute medians is too high.
- Have to live with worst case $\theta(N^2)$ if we want good practical performance.

1 : 2 with Mergesort as the winner.

31.2 Quick Select

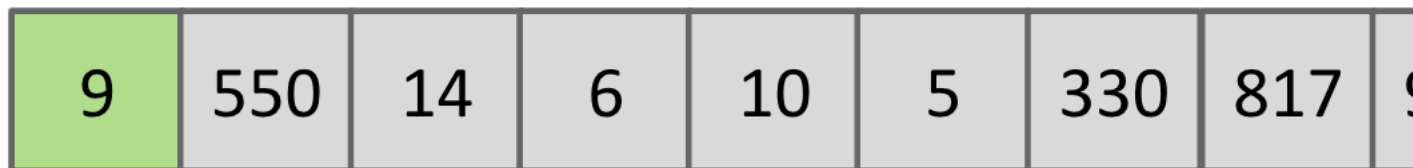
Dissatisfied with the result that Quicksort is unable to completely defeat Mergesort and truly claim the title for “fastest comparison-based sorting” algorithm, let’s shoot our one last shot to overturn the result: use Quick Select to identify the median.

Quick Select Algorithm

Goal: find the median using partitioning.

Key idea: Median of an length n array will be around index $n/2$.

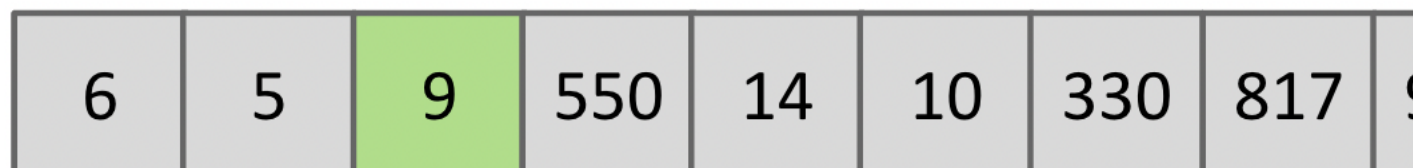
1. Initialize array with the leftmost item as the pivot.



Initial Array

For this example, the index of the median should be $9 / 2 = 4$.

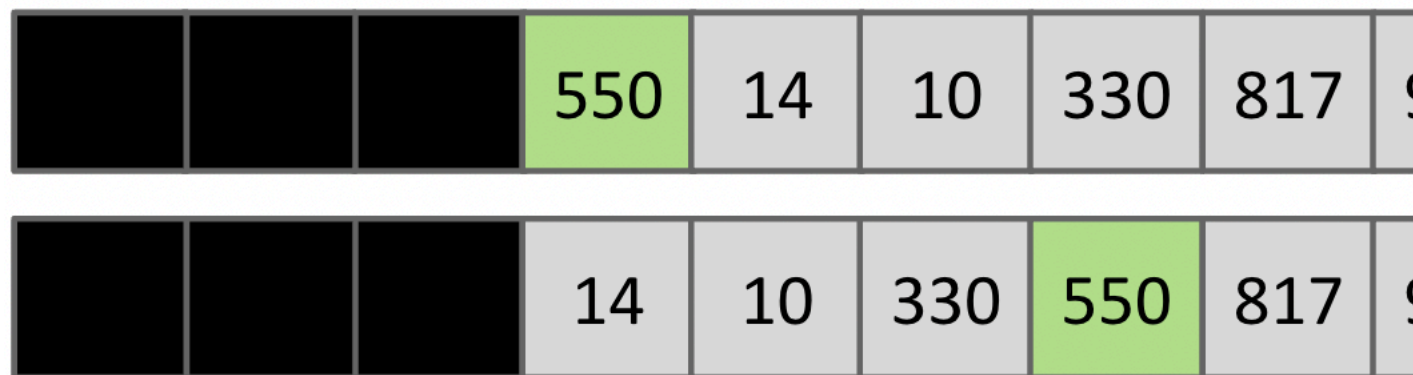
2. Partition around pivot.



Pivot lands at index 2

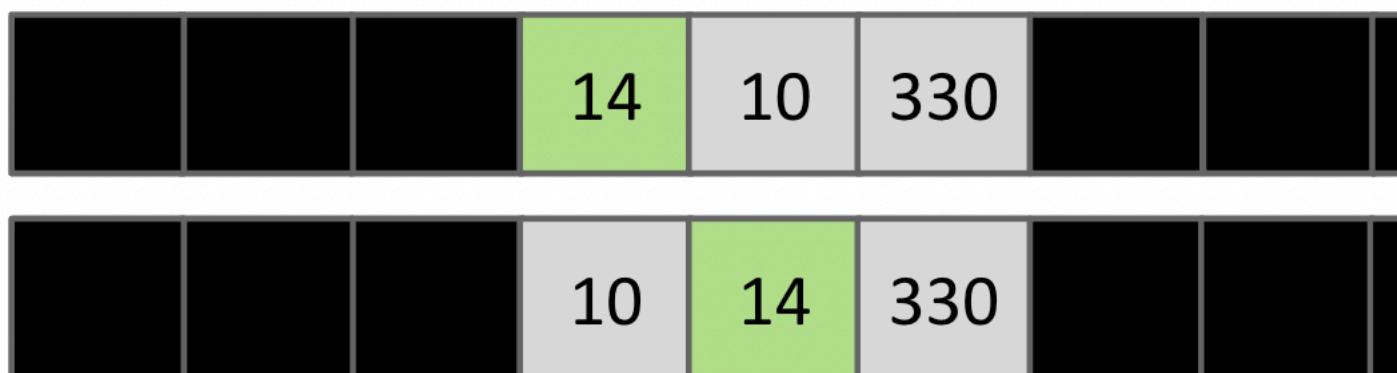
Since the pivot is at index 2, which is not the median. Therefore, need to continue. However, will only partition the **right** subproblem because median can't be to the left! (index $n/2 > 2$)

3. Partition the subproblem. Repeat the process.



Pivot is at index 6, which means that it is not at the median. Continue.

4. Stop when the pivot is at the median index.



Since pivot is at index 4, we are done.

Quick Select Performance

Worst Case Runtime

The worst case is when the array is in sorted order, which yields a runtime of $\theta(N^2)$.

[1 2 3 4 5 6 7 8 9 10 ... N]

[1 **2** 3 4 5 6 7 8 9 10 ... N]

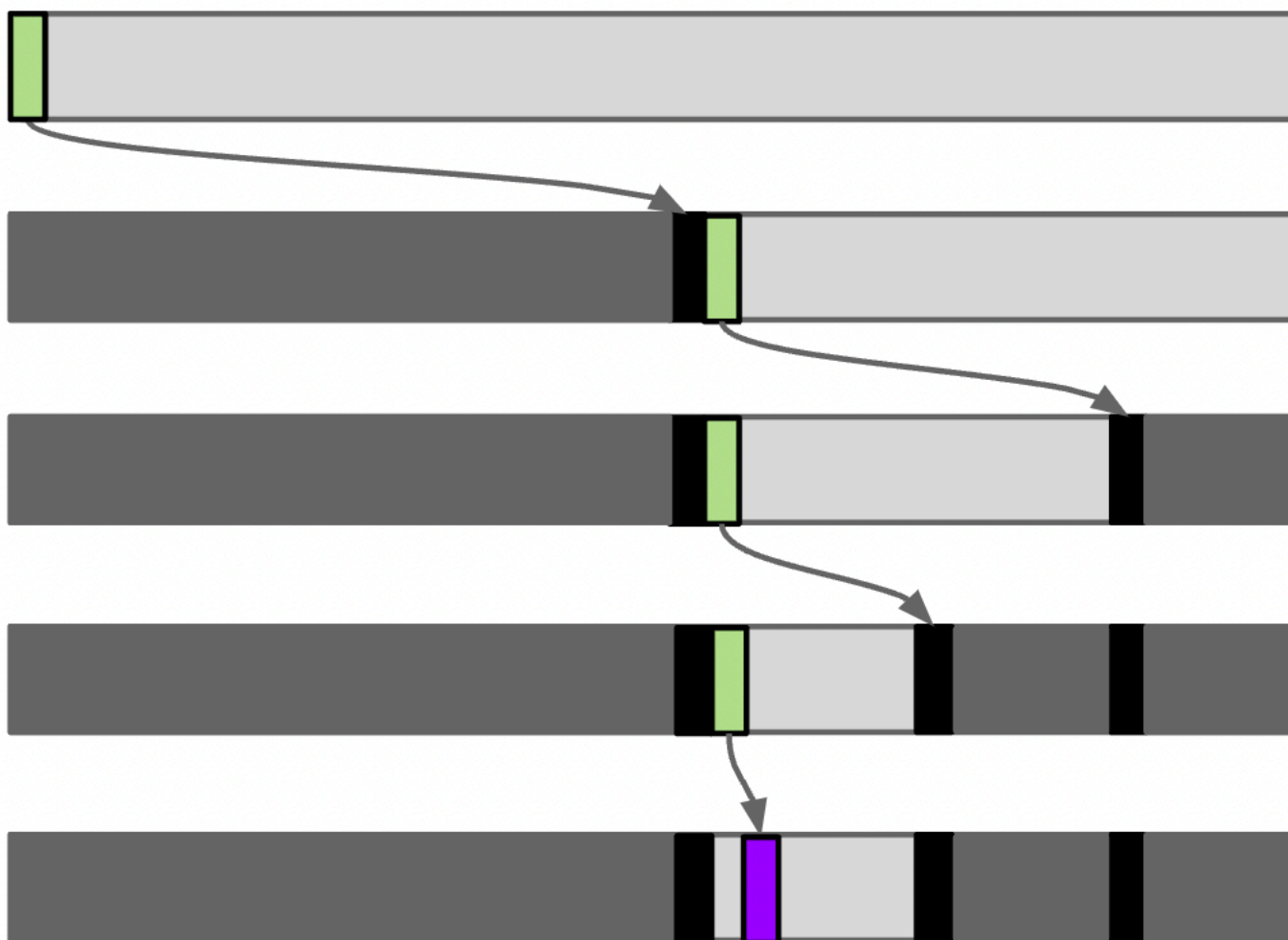
[1 2 **3** 4 5 6 7 8 9 10 ... N]

...

[1 2 3 4 5 ... **N/2** ... N]

Expected Runtime

The expected runtime for the Quick Select Algorithm is $\theta(N)$. Here's an intuitive diagram that justifies this result:



Using one of our favorite sums, we can see the runtime is:

$$N + N/2 + N/4 + \dots + 1 = \theta(N)$$

Compared to MergeSort?

Unfortunately, even using Quickselect to find the exact median, the resulting algorithm is still quite slow.

Team Mergesort wins, sadly.

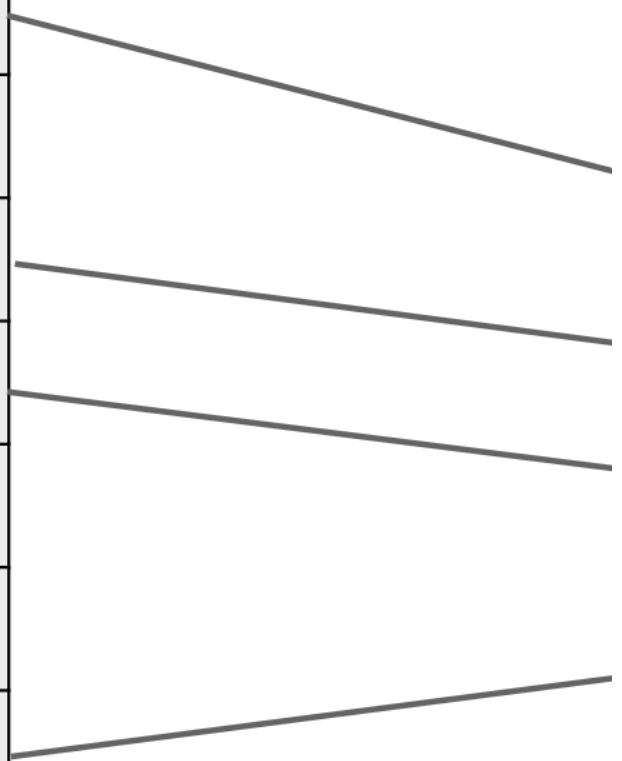
31.3 Stability, Adaptiveness, and Optimization

Stability

A sort is stable if the order of equivalent elements is preserved. The following is an example of a stable sort. After sorting by section, notice how Bas, Jana, Jouni, and Rosella are in the same order as before sorting. If we want records sorted by section and then by name within each section, we can sort by name and then by section as below.

`sort(studentRecords, BY_NAME);`

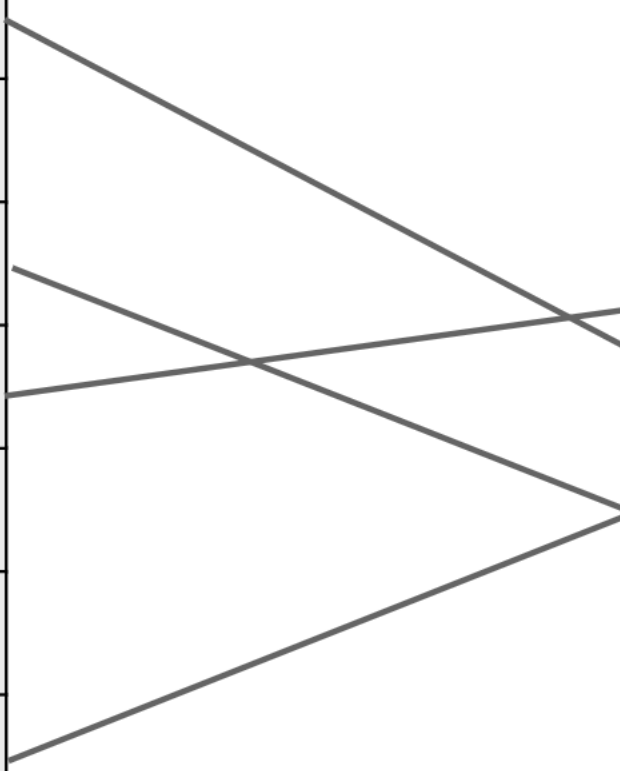
Bas	3
Fikriyya	4
Jana	3
Jouni	3
Lara	1
Nikolaj	4
Rosella	3
Sigurd	2



The following example is an unstable sort. It can make things really annoying! If we want records sorted by section and then by name within each section, we can't just sort by name and then by section as before. After an unstable sort, the previous ordering is not maintained.

`sort(studentRecords, BY_NAME);`

Bas	3
Fikriyya	4
Jana	3
Jouni	3
Lara	1
Nikolaj	4
Rosella	3
Sigurd	2



Are some of the sorts we learned stable? Insertion sort is stable! Equivalent elements move past their equivalent brethren. MergeSort is stable. HeapSort is not stable. QuickSort can be stable depending on its partitioning scheme, but its stability cannot be assumed since many of its popular partitioning schemes, like Hoare, are unstable.

Optimizations

Adaptiveness - A sort that is adaptive exploits the existing order of the array. Examples are InsertionSort, SmoothSort, and TimSort.

Switch to Insertion Sort - When a subproblem reaches size 15 or lower, use insertion sort. It is very very fast for inputs of small sizes.

Exploit restrictions on set of keys - For example, if the number of keys is some constant, we can use this constraint to sort faster by applying 3-way QuickSort.

Switch from QuickSort - If the recursion goes too deep, switch to a different type of sort.

31.4 Summary

Quicksort

What can we do to avoid the worst case runtime of $\theta(N^2)$ for Quicksort?

1. Randomness: pick a random pivot point, shuffle items before sorting
2. Smarter Pivot Selection: constant time pivot pick (ex: pick a few and then choose the best out of them), linear time pivot pick like (ex: median)
3. Introspection: switch to a different sort if current sort hits recursion depth threshold

Quicksort vs. Mergesort

- Mergesort is faster than Quicksort L3S (leftmost pivot, 3-scan partition) and QuickSort PickTH (median pivot, Tony Hoare partition)
- Quicksort LTHS (leftmost pivot, Tony Hoare partition) is faster than Mergesort!
- Tony Hoare's partitioning:
 - two pointers, one at each end of the items array, walking towards each other
 - left pointer hates larger or equal items, right pointer hates smaller or equal items
 - swapping anything they don't like; stop when the two pointers cross each other
 - new pivot = Item at right pointer.

Quick Select

Quick select helps us quickly find the median with partitioning. Median of an length n array will be around index $n/2$.

1. Initialize array with the leftmost item as the pivot.
2. Partition around pivot.
3. Partition the subproblem. Repeat the process.
4. Stop when the pivot is at the median index.

Expected runtime: $\theta(N)$. Worst case runtime (when array is in sorted order): $\theta(N^2)$.

Stability, Adaptiveness, and Optimization

Stability: A sort is stable if the order of equivalent elements is preserved.

Optimizations:

- Adaptiveness - sort that exploits the existing order of the array.
- Switch to Insertion Sort - when a subproblem reaches size 15 or lower
- Exploit restrictions on set of keys
- Switch from QuickSort - if the recursion goes too deep

31.5 Exercises

Factual

1. Suppose we have the array [17, 15, 19, 32, 2, 26, 41, 17, 17], and we partition it using 3-scan partitioning using 17 as the pivot. What array do we end up with?
2. Suppose we have the array [17, 15, 19, 32, 2, 26, 41, 17, 17], and we partition it using Tony Hoare partitioning using 17 as the pivot. What array do we end up with?

- Problem 1
- Problem 2

Conceptual

1. Which of the following arrays will result in N^2 behavior if we use quicksort which always uses the leftmost item as pivot, and which uses a stable partitioning strategy?
 - ☐ [1, 1, 1, 1, 1, 1, ..., 1]
 - ☐ [1, 2, 3, 4, 5, 6, ..., N]
 - ☐ [N, N - 1, N - 2, N - 3, ..., 2, 1]
2. Repeat the exercise above, but assume that we shuffle the array before partitioning.
3. Suppose we try to use Quick Select to find the median of [17, 15, 19, 32, 2, 26, 41, 5, 9]. How many total partitioning operations will we need to complete to find the median?
4. Which of the following sorts are stable?
 - ☐ Insertion sort
 - ☐ Quicksort L3 - use leftmost item as pivot, use 3scan partitioning
 - ☐ Quicksort L3S - use leftmost item as pivot, use 3scan partitioning, shuffle before starting
 - ☐ Quicksort LTH - use leftmost item as pivot, use tony hoare partitioning
 - ☐ Quicksort PickTH - use median (computed with PICK) as pivot, use tony hoare partitioning

- Problem 1
- Problem 2
- Problem 3
- Problem 4

Metacognitive

1. Why does Java's built-in `Array.sort` method use Quicksort for `int`, `long`, `char`, or other primitive arrays, but Mergesort for all `Object` arrays?
2. Given that quicksort runs fastest if we can always somehow pick the median item as the pivot, why don't we use quickselect to find the median to optimize our pivot selection (as opposed to using the leftmost item)?

- Problem 1
- Problem 2

31. More Quick Sort, Sorting Summary